

Autonomous Kernel Observability: Integrating eBPF-Driven System Call Sequencing with Local Large Language Models for Intent-Based Zero Trust Enforcement

The modern cybersecurity landscape is increasingly defined by the struggle between complex, multi-stage attacks and the limitations of traditional, reactive monitoring frameworks. Advanced Persistent Threats (APTs) often manipulate the system in subtle ways that appear benign in isolation but reveal malicious intent when viewed as a sequence of actions over time.¹ Traditional host-based intrusion detection systems (HIDS) frequently suffer from either high performance overhead or a lack of semantic context, failing to bridge the gap between what a process is doing and why it is doing it.³ The emergence of extended Berkeley Packet Filter (eBPF) technology offers a revolutionary path forward, enabling deep, high-performance observability directly within the Linux kernel.⁵ By coupling eBPF's low-level tracing capabilities with the cognitive power of local Large Language Models (LLMs), security architects can now implement a ZeroTrustAgent capable of identifying intent-based anomalies with unprecedented fidelity.⁷

Foundations of Kernel-Level Observability via eBPF

To understand the shift toward intent-based monitoring, one must first analyze the mechanical advantages of eBPF over legacy instrumentation techniques such as ptrace or standard audit logging via auditd. Legacy systems often incur heavy context-switching costs, as the CPU must frequently transition between kernel and user space to deliver event data.² Furthermore, the /proc filesystem, while ubiquitous, is inefficient for real-time monitoring because it requires opening, reading, and parsing text files for every check, a process that is both rigid and slow.⁹

The eBPF Virtual Machine and Safety Verifier

eBPF transforms the Linux kernel into a programmable platform by allowing sandboxed programs to run at specific hook points.⁵ The architecture revolves around a restricted, in-kernel virtual machine with its own instruction set.¹¹ Before an eBPF program is allowed to execute, it must pass through a rigorous verifier that performs static analysis to ensure the program cannot crash the kernel, access unauthorized memory, or enter infinite loops.⁸ This safety model allows eBPF programs to run with near-native performance after being Just-In-Time (JIT) compiled into machine-specific instructions.¹⁰

Feature	Legacy Auditing (auditd, /proc)	eBPF-Based Monitoring
Execution Context	User-space processing	In-kernel execution
Performance Overhead	High (Context switches)	Minimal (Near-native JIT)
Safety	High (User-space isolation)	Guaranteed (Kernel Verifier)
Programmability	Rigid, configuration-based	Fully programmable C-like code
Data Delivery	Periodic polling/text logs	Real-time binary ring buffers

The ability to attach programs to various hook points—such as system calls (kprobes), tracepoints, and network events (XDP)—enables the collection of high-fidelity data that was previously inaccessible without risky kernel modifications.⁵ For intent-based anomaly detection, the system must focus on the primary interfaces where processes interact with the outside world: process execution, file access, and network communication.³

Technical Objective: Intent-Based Monitoring Architecture

The primary technical objective is the implementation of a ZeroTrustAgent that captures sequences of `execve`, `connect`, and `openat` system calls. These specific syscalls are the building blocks of most malicious activities, including lateral movement, data exfiltration, and credential theft.¹ The architecture follows a multi-stage pipeline where raw kernel events are transformed into semantic narratives for a local LLM to classify.

The Semantic Gap and Behavioral Intent

Traditional security tools often operate on a "signature" basis, looking for known malicious binary hashes or IP addresses. However, as AI agents and complex scripts become more

prevalent, the "semantic gap" between intent and action widens.³ An AI agent might receive a command to "refactor code," but an indirect prompt injection attack could cause it to plant a backdoor.³ To identify such anomalies, the monitor must observe not just the individual syscalls, but the causal dependencies and the "why" behind the sequence.⁴

By monitoring at the system boundaries—the kernel-user interface—eBPF allows the system to remain framework-agnostic. Whether an agent is built with LangChain, AutoGen, or raw Python, it must ultimately use the kernel to perform actions.³ This makes the kernel the "ground truth" of system behavior.¹³

BPF Program Implementation: The Kernel Probe

The first required component is a C-based eBPF program that hooks into the target system calls. This program must efficiently extract arguments from the syscall context and push them to a perf_buffer for user-space processing.¹⁴

Hooking execve for Process Provenance

The execve syscall is triggered whenever a new program is executed. Capturing the parent PID, the process name, and the filename of the executed binary is essential for building a process tree.¹⁶

C

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>
```

```
struct exec_event_t {
    u32 pid;
    u32 ppid;
    char comm;
    char filename;
};
```

```
BPF_PERF_OUTPUT(exec_events);
```

```
int kprobe__sys_execve(struct pt_regs *ctx, const char __user *filename) {
    struct exec_event_t event = {};
    u64 pid_tgid = bpf_get_current_pid_tgid();

    event.pid = pid_tgid >> 32;
```

```

// Parent PID is found in the current task_struct
struct task_struct *task = (struct task_struct *)bpf_get_current_task();
event.ppid = task->real_parent->tgid;

bpf_get_current_comm(&event.comm, sizeof(event.comm));
// Safely read the filename from user space memory
bpf_probe_read_user_str(&event.filename, sizeof(event.filename), filename);

exec_events.perf_submit(ctx, &event, sizeof(event));
return 0;
}

```

The use of `bpf_probe_read_user_str` is a critical safety requirement. Because eBPF programs cannot directly dereference pointers into user-space memory, they must use this helper to copy the string into a kernel-controlled buffer.⁶ Failure to do so would result in the verifier rejecting the program.¹⁰

Auditing File Access and Network Activity

To track "effects," the monitor hooks `openat` and `tcp_v4_connect`. Monitoring `openat` allows the system to identify which files are being modified or read, while `tcp_v4_connect` provides insight into external communications.⁵

Syscall	Parameters Captured	Purpose
<code>openat</code>	Filename, flags (<code>O_RDONLY</code> , <code>O_WRONLY</code>)	Detect unauthorized file access or data modification
<code>tcp_v4_connect</code>	Destination IP, Destination Port	Identify C2 server communication or lateral movement
<code>execve</code>	Binary path, command-line arguments	Monitor program lifecycles and script executions

In the `tcp_v4_connect` hook, the program accesses the `sock` structure to extract IP addresses. This is a common technique used in tools like `tcpconnect` from the BCC toolkit to trace active

TCP connections.⁵

Python Wrapper: The BCC-Based Orchestrator

The second component is the Python wrapper, which utilizes the BPF Compiler Collection (BCC) to load the C program, manage maps, and poll the perf buffer.¹⁹

Sequence Batching and PID Tracking

A fundamental challenge in syscall-based anomaly detection is the sheer volume of data. A typical Linux system can generate thousands of syscalls per second. To make this data digestible for an LLM, the Python wrapper must perform sequence batching based on the Process ID (PID) or Thread ID (TID).²¹

The wrapper maintains an associative array where each PID's syscall events are stored in a rolling window. Once a sequence reaches a predefined length (e.g., 20 events) or a critical event like a connection to a suspicious IP occurs, the sequence is serialized into a textual format for inference.²³

Python

```
from bcc import BPF
import ctypes as ct

# Define the data structure in Python to match the C struct
class ExecEvent(ct.Structure):
    _fields_ = [
        ("pid", ct.c_uint32),
        ("ppid", ct.c_uint32),
        ("comm", ct.c_char * 16),
        ("filename", ct.c_char * 256),
    ]

b = BPF(text=bpf_program_c_code)

def print_exec_event(cpu, data, size):
    event = ct.cast(data, ct.POINTER(ExecEvent)).contents
    # Add to PID-specific sequence buffer
    process_buffer[event.pid].append(f"exec {event.filename.decode()}")
    if len(process_buffer[event.pid]) > 20:
        analyze_sequence(process_buffer[event.pid])
```

```
b["exec_events"].open_perf_buffer(print_exec_event)
while True:
    b.perf_buffer_poll()
```

The use of BPF_PERF_OUTPUT is the preferred method for pushing per-event data to user space because it utilizes a per-CPU ring buffer, which avoids lock contention in multi-core systems.¹⁴

Inference Engine: Local LLM Classification

The third component is the integration with a quantized local LLM, such as Llama-3-8B-GGUF, using llama-cpp-python.²⁵ This choice ensures that sensitive system telemetry never leaves the host, satisfying privacy and compliance requirements.²⁶

Model Selection and Quantization (GGUF)

For a real-time monitor, the model must be balanced between reasoning capability and inference speed. Llama-3-8B, when quantized to 4-bit (Q4_K_M) or 5-bit (Q5_K_M), provides an excellent middle ground.²⁷ Quantization reduces the model size and computational demands, enabling fast inference on CPUs without requiring high-end GPUs.²⁵

The llama.cpp runtime is engineered for efficiency, utilizing memory mapping (mmap) to load only necessary parts of the model and supporting various hardware acceleration backends such as Metal for Apple Silicon and CUDA for NVIDIA GPUs.²⁵

Serialization and Prompt Engineering for Anomaly Detection

To identify "Intent-based" anomalies, the sequence of syscalls must be serialized into a narrative that the LLM can interpret.²⁴ This is more effective than feeding raw integers, as the LLM can leverage its pre-trained understanding of system administration and software patterns.³

A typical prompt might look like:

"System Activity Log:

1. Process 'bash' (PID 450) executed.
2. 'bash' opened '/etc/shadow' for reading.
3. 'bash' connected to 104.21.43.12 on port 4444.
4. 'bash' executed '/tmp/sh'.

Task: Classify this behavior as 'BENIGN' or 'MALICIOUS' based on the intent of a standard system shell. Provide a reason."

The model's output can be constrained using JSON schema mode to ensure the ZeroTrustAgent can programmatically parse the decision.³³ The use of structured outputs

minimizes the risk of hallucinations and ensures the decision can be acted upon in real-time.³³

Dataset Analysis: DARPA Transparent Computing and Syzbot

Training and validating the ZeroTrustAgent requires access to realistic provenance data. Two primary sources are the DARPA Transparent Computing (TC) datasets and Syzbot crash logs.³⁴

The Common Data Model (CDM) and Provenance Graphs

The DARPA TC datasets contain fine-grained provenance information, including process executions and file accesses, generated by red teams in live environments.¹ This data is represented using the Common Data Model (CDM), which defines entities (processes, files, sockets) as nodes and events (syscalls) as directed, labeled edges.¹

CDM Entity	Key Fields	Anomaly Detection Value
Subject	cmdLine, privilegeLevel, ppid	Provides actor context for the LLM
Event	type, timestampNanos, sequence	Establishes the causal chain of events
FileObject	path, hashes, size	Identifies the target of malicious intent
NetFlow	remoteAddress, remotePort, protocol	Tracks data exfiltration or C2 patterns

Analyzing these datasets reveals pronounced spikes in specific events during attack windows, such as SHELL-COMMAND and PROCESS-CREATE aligned with lateral movement.¹ The ZeroTrustAgent can use these patterns as few-shot examples in its prompt to improve classification accuracy.³⁶

Syzbot and Syscall Dependency Relations (SDRs)

Syzbot logs provide a different type of data: sequences of syscalls that trigger kernel crashes.³⁵ These logs are crucial for identifying patterns that indicate exploitation of kernel-level vulnerabilities.³⁵ Research into "Psyzkaller" shows that kernel fuzzers can be improved by learning Syscall Dependency Relations (SDRs) from both normal and crash-inducing traces.³⁵ For the ZeroTrustAgent, these SDRs define the boundaries of "syntactic correctness" for system behavior.

Zero Trust Enforcement: Blocking Malicious Intent

A true ZeroTrustAgent must not only observe but also enforce policies. Once the LLM identifies a malicious intent, the agent must intervene at the kernel level to prevent the attack from progressing.⁷

Preventing Lateral Movement with `bpf_send_signal`

The `bpf_send_signal()` helper function allows an eBPF program to send a Unix signal, such as SIGKILL, to the current process.³⁸ This is far more effective than user-space blocking because it occurs immediately after the suspicious syscall is detected, potentially before the syscall even returns.³⁸

A common scenario for intervention is the detection of a privilege escalation sequence: `fork`, `setuid(0)`, and then `execve` to spawn a root shell.³⁸ By tracking the state of a PID in an eBPF hash map, the program can identify when a process has successfully gained root and is attempting to execute a sensitive binary.³⁸

Identity-Driven Network Security

Beyond process termination, the ZeroTrustAgent can implement network policies that are identity-aware rather than IP-aware.⁷ By integrating with Cilium-style identities, the agent can use eBPF to drop packets from any process that deviates from its "intent profile".⁷ For example, if a process labeled as "Front-End-Web" attempts to connect to a management port on a database server, the eBPF program can block the connection at the XDP (Express Data Path) layer, which processes packets before the kernel even allocates memory for them, resulting in sub-100ns latency.¹⁰

Performance and Reliability Analysis

Implementing an AI-driven security agent in the critical path of the kernel requires careful optimization to avoid becoming a bottleneck.

Minimizing Overhead with Speculative Decoding and Batching

To maintain a performance overhead of less than 3%, the system must use asynchronous inference.³ The eBPF monitor pushes data to the perf buffer, and the LLM processes it in a separate thread. For high-volume systems, "Dynamic Batching" can be used to accumulate

multiple inference requests and process them as a single batch, maximizing hardware utilization.⁴¹

Speculative decoding can further reduce latency by using a small "draft" model to predict common benign sequences (e.g., standard apt update behavior) and only invoking the larger Llama-3-8B model when the draft model indicates a potential anomaly.²⁹

Optimization Technique	Mechanism	Impact
Quantization (GGUF)	Reduces weight precision (4-bit/5-bit)	Lowers RAM usage by >50%
In-Kernel Filtering	Filters common benign syscalls in eBPF	Reduces data volume to user space
Dynamic Batching	Processes multiple PIDs in one LLM pass	Increases token throughput
Speculative Decoding	Uses a draft model for fast verification	Shaves milliseconds off inference

The Role of the Verifier in Production Stability

One of the most significant advantages of eBPF is that it cannot "crash" the kernel.¹⁰ In a production environment, this is critical. Traditional kernel modules can lead to kernel panics if they contain bugs. In contrast, if an eBPF program attempts to perform an unsafe operation, the verifier will simply refuse to load it.⁸ This makes eBPF-based security tools suitable for mission-critical infrastructure where stability is as important as security.¹⁰

Conclusions and Future Directions

The integration of eBPF and local LLMs marks the beginning of a new era in system security. By moving beyond simple signature matching and embracing intent-based analysis, the ZeroTrustAgent provides a deep, semantic layer of protection that is difficult for attackers to bypass.² The use of `execve`, `connect`, and `openat` syscalls as the foundation for this monitoring

ensures that the most critical system actions are always under scrutiny.

Future developments in this field will likely focus on "Closed-Loop AI Control," where the system not only detects and blocks threats but also autonomously adapts its policies based on evolving behavior patterns.⁸ The expansion of eBPF hooks into the Linux Security Module (LSM) layer will allow for even more granular control, such as blocking specific file path patterns or network protocols dynamically.⁴² As local inference hardware continues to improve, the latency gap will narrow, eventually allowing for near-real-time, intent-aware blocking for every system call in the Linux environment. This evolution toward self-observing and self-protecting platforms will be a cornerstone of modern cybersecurity architecture.⁸

Works cited

1. DARPA Transparent Computing Program - Emergent Mind, accessed April 26, 2026, <https://www.emergentmind.com/topics/darpa-transparent-computing-program>
2. Beginner's Guide to AI Security with eBPF - Palo Alto Networks Blog, accessed April 26, 2026, <https://www.paloaltonetworks.com/blog/network-security/beginners-guide-to-ai-security-with-ebpf/>
3. AgentSight: System-Level Observability for AI Agents Using eBPF - arXiv, accessed April 26, 2026, <https://arxiv.org/html/2508.02736v1>
4. AgentSight: Keeping Your AI Agents Under Control with eBPF-Powered System Observability | by yunwei37 | Medium, accessed April 26, 2026, <https://medium.com/@yunwei356/agentsight-keeping-your-ai-agents-under-control-with-ebpf-powered-system-observability-6db90abec6e1>
5. eBPF — Coding the kernel without coding the kernel! | by Rishub ..., accessed April 26, 2026, <https://medium.com/@rishub/ebpf-coding-the-kernel-without-coding-the-kernel-e4d341adc043>
6. The beginning of my eBPF Journey — Kprobe Adventures with BCC | by Douglas Mendez, accessed April 26, 2026, <https://douglassmakey.medium.com/the-beginning-of-my-ebpf-journey-kprobe-adventures-with-bcc-6aae3eb33a1e>
7. Zero Trust Networking - Cilium, accessed April 26, 2026, <https://cilium.io/outcomes/zero-trust/>
8. eBPF Unlocked Redefining Kernel Observability, Security, and AI - Happiest Minds, accessed April 26, 2026, <https://www.happiestminds.com/wp-content/uploads/2026/03/eBPF-Unlocked-Redefining-Kernel-Observability-Security-and-AI.pdf>
9. eBPF Tutorial: BPF Iterators for Kernel Data Export | by yunwei37 | Medium, accessed April 26, 2026, <https://medium.com/@yunwei356/ebpf-tutorial-bpf-iterators-for-kernel-data-export-447cc8dbc63d>
10. eBPF: The Most Powerful Technology in Your Linux Kernel You've ..., accessed

April 26, 2026,

<https://medium.com/@moksh.9/ebpf-the-most-powerful-technology-in-your-linux-kernel-youve-never-written-ff9a5688b669>

11. Building usefull tools with eBPF: Part 1, Setting up bcc | Prathyush's Blog, accessed April 26, 2026,
https://prathyushpv.github.io/2019/05/20/Building_usefull_tools_with_eBPF_Part1_Setting_up_bcc.html
12. eBPF-Based Approach to Tracing System Calls and Predicting Privilege Escalation Attacks, accessed April 26, 2026,
https://www.inf.ufrgs.br/~granville/wp-content/papercite-data/pdf/bertinato2024_2.pdf
13. GitHub - darpa-i2o/Transparent-Computing, accessed April 26, 2026,
<https://github.com/darpa-i2o/transparent-computing>
14. bcc Reference Guide - Android GoogleSource, accessed April 26, 2026,
https://android.googlesource.com/platform/external/bcc/+685ec23e/docs/reference_guide.md
15. The beginning of my eBPF Journey - Kprobe Adventures with BCC, accessed April 26, 2026,
<https://www.kungfudev.com/blog/2023/10/14/the-beginning-of-my-ebpf-journey-kprobe-bcc>
16. How to Track Process Lifecycle Events with eBPF - OneUptime, accessed April 26, 2026,
<https://oneuptime.com/blog/post/2026-01-07-ebpf-process-lifecycle-tracking/view>
17. An Event-based Data Model for Granular Information Flow ... - USENIX, accessed April 26, 2026, <https://www.usenix.org/system/files/tapp2020-paper-khoury.pdf>
18. iovisor/bcc: BCC - Tools for BPF-based Linux IO analysis, networking, monitoring, and more - GitHub, accessed April 26, 2026, <https://github.com/iovisor/bcc>
19. Surveying the eBPF Developer Ecosystem | Kevin Sookocheff, accessed April 26, 2026,
<https://sookocheff.com/post/ebpf/surveying-the-ebpf-developer-ecosystem/>
20. eBPF For Humans | 02 | BCC. In this session, we will practice and... | IlayGilman, accessed April 26, 2026,
<https://medium.com/@ilaygilman/ebpf-for-humans-02-bcc-3d96333adc9b>
21. Efficient Approach for Anomaly Detection in IoT Using System Calls - PMC, accessed April 26, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9861298/>
22. How to Trace System Calls and Functions with eBPF (bpfftrace, bcc) - OneUptime, accessed April 26, 2026,
<https://oneuptime.com/blog/post/2026-01-07-ebpf-tracing-bpfftrace-bcc/view>
23. Anomaly Detection of System Call Sequence Based on Dynamic Features and Relaxed-SVM - ResearchGate, accessed April 26, 2026,
https://www.researchgate.net/publication/358279062_Anomaly_Detection_of_System_Call_Sequence_Based_on_Dynamic_Features_and_Relaxed-SVM
24. LTAAnomaly: A Transformer Variant for Syslog Anomaly Detection Based on Multi-Scale Representation and Long Sequence Capture - MDPI, accessed April

- 26, 2026, <https://www.mdpi.com/2076-3417/13/13/7668>
25. How to Use llama.cpp to Run LLaMA Models Locally - Codecademy, accessed April 26, 2026, <https://www.codecademy.com/article/llama-cpp>
 26. Building a Local AI Assistant with llama-cpp-python | Ruan Bekker's ..., accessed April 26, 2026, <https://ruan.dev/blog/2025/02/14/building-a-local-ai-assistant-with-llama-cpp-python>
 27. llama.cpp: Fast Local LLM Inference, Hardware Choices & Tuning - Clarifai, accessed April 26, 2026, <https://www.clarifai.com/blog/llama.cpp>
 28. llama.cpp: The Ultimate Guide to Efficient LLM Inference and Applications - PyImageSearch, accessed April 26, 2026, <https://pyimagesearch.com/2024/08/26/llama-cpp-the-ultimate-guide-to-efficient-llm-inference-and-applications/>
 29. llama.cpp: The Lightweight Engine Behind Local LLMs - Sandgarden, accessed April 26, 2026, <https://www.sandgarden.com/learn/llama-cpp>
 30. Getting Started - llama-cpp-python, accessed April 26, 2026, <https://llama-cpp-python.readthedocs.io/en/stable/>
 31. AnOLLM: Large Language Models for Tabular Anomaly Detection - OpenReview, accessed April 26, 2026, <https://openreview.net/forum?id=7VkhffT5X2>
 32. From Commands to Prompts: LLM-based Semantic File System - arXiv, accessed April 26, 2026, <https://arxiv.org/html/2410.11843v4>
 33. Structured outputs with llama-cpp-python, a complete guide w - Instructor, accessed April 26, 2026, <https://python.useinstructor.com/integrations/llama-cpp-python/>
 34. Semantic-Aware Advanced Persistent Threat Detection Using Autoencoders on LLM-Encoded System Logs - arXiv, accessed April 26, 2026, <https://arxiv.org/html/2602.00204v1>
 35. Pszykaller: Learning from Historical and On-the-Fly Execution Data for Smarter Seed Generation in OS kernel Fuzzing - arXiv, accessed April 26, 2026, <https://arxiv.org/html/2510.08918v1>
 36. Examples of Prompts | Prompt Engineering Guide, accessed April 26, 2026, <https://www.promptingguide.ai/introduction/examples>
 37. Best Prompt Techniques for Best LLM Responses | by Jules S. Damji | The Modern Scientist, accessed April 26, 2026, <https://medium.com/the-modern-scientist/best-prompt-techniques-for-best-llm-responses-24d2ff4f6bca>
 38. bpf_send_signal | Engineering Everything with eBPF, accessed April 26, 2026, https://ebpf.hamza-megahed.com/docs/chapter5/4-bpf_send_signal/
 39. Using bpf_send_signal to Terminate Malicious Processes in eBPF ..., accessed April 26, 2026, <https://eunomia.dev/tutorials/25-signal/>
 40. [Show] I built a Zero Trust Network Controller using eBPF/XDP : r/rust - Reddit, accessed April 26, 2026, https://www.reddit.com/r/rust/comments/1qy3mqi/show_i_built_a_zero_trust_network_controller/
 41. Dynamic Batching with Llama 3 8B with Llama.cpp CPUs Tutorial, accessed April

26, 2026,

<https://docs.wallaroo.ai/wallaroo-llm/wallaroo-llm-tutorials/wallaroo-llm-optimizations-tutorials/wallaroo-llm-deploy-llamacpp-dynamic-batching-tutorial/>

42. KFunc 'bpf_send_signal_task' - eBPF Docs, accessed April 26, 2026,

https://docs.ebpf.io/linux/kfuncs/bpf_send_signal_task/